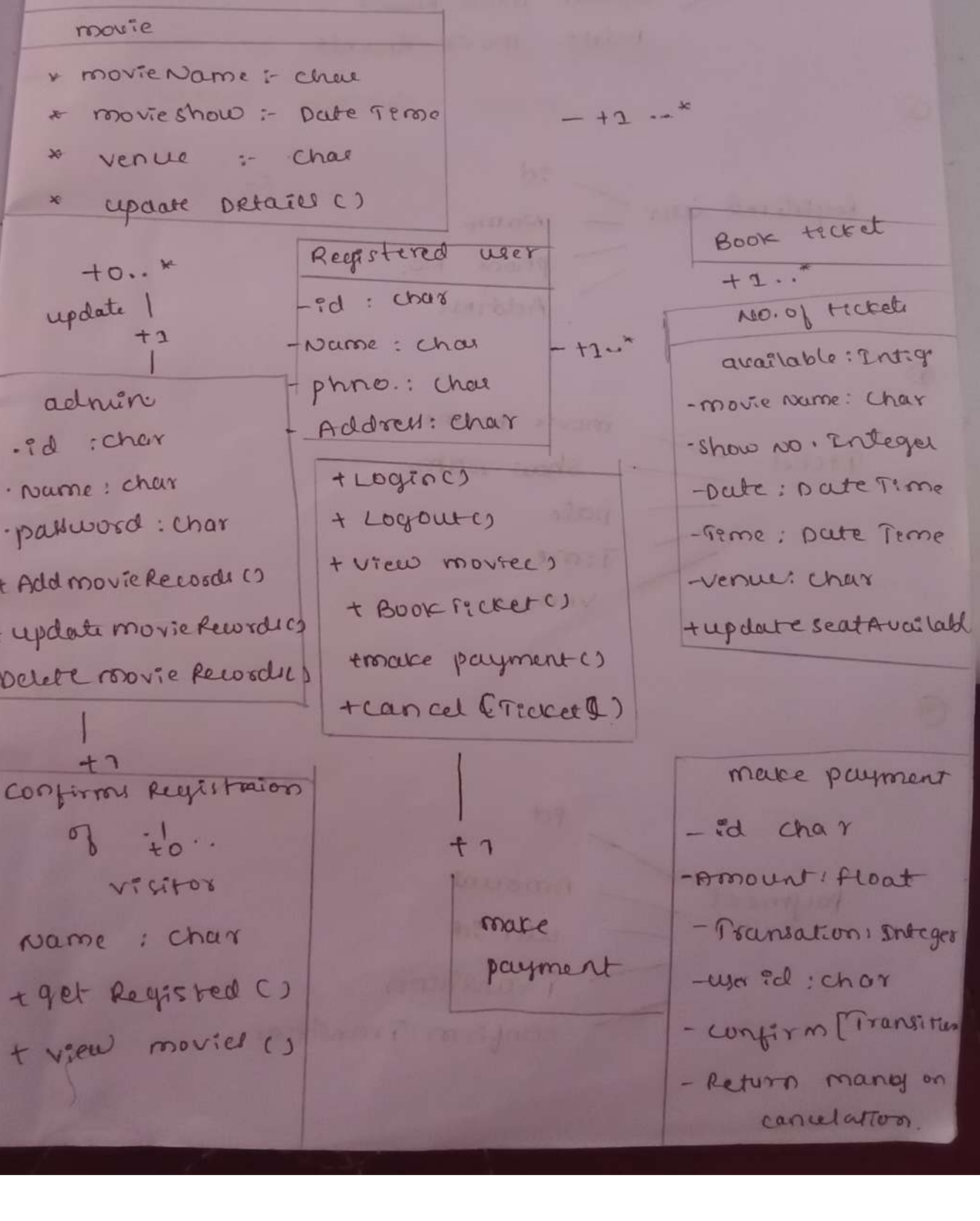
software engineering :-

UML diagram (Book my show)

Ticket booking

* movie
* Registered user
* Book ticket
* visitor
* admin
* make payment



* Domain & attributes :-

① movie
- name
- show
- venue

②

admin
- id
- password
- name
- update movie Record
- Delete movie Records

③

Registered user
- id
- Name
- phone No.
- Address

④

Ticket
- movie Name
- Show Name
- Date
- Time
- Venue

⑤

Payment
- id
- Amount
- user id
- Transition.
- confirm Transition.

~~Title : Stakeho~~

Experiment - 02

Aim :- To identify entities & attributes & relationship & represents them.

objectives :- To understand domain modeling concepts & create a domain dia for a given case study

domains & attributes

① Users
- * User_ID
- * Name
- * Email
- * Phone

② Ticket
- * Ticket_ID
- * Seat_Number
- * Ticket_Status

③ Bookings
- * Booking_ID
- * Booking_Date

④ payment
- * payment_ID
- * payment_method
- * payment_date

5 schedule
- * schedule_ID
- * Date
- * Time

⑥ Admin

* admin Id
* Email
* password

main classes

* customer
* Ticket
* Booking
* payment
* admin

4
26/3

Title :- Stakeholder Identification & Requirements Analysis

Task 1 : Stakeholder Identification

Primary Stakeholder :

Customers, End users, System Administrators, customer support staff

Secondary stakeholders :

Service providers, payment Gateway providers marketing & Sales Teams, IT & maintenance Team

External stakeholders :

Regulatory Authorities, Bank & financial institution, Government & Tax Authorities

Regulatory stakeholders :

Data protection, privacy Authorities, Financial & Payment Regulatory Bodies, customer protection Authorities, Tax Authorities.

Task 2 : Stakeholder Needs Analysis.

① Explicit needs :

Stakeholders	Explicit need
→ customer	Search Ticket
→ customer	online payment
→ Administrator	manage schedules
→ payment Gateways	Refund processing.

② Implicit needs :-

Stakeholders	Implicit needs
→ customer	Data security & Fast performance
→ Administrator	Data Accuracy
→ Tax Authorities	proper Tax Handling (G...
→ Service provider	Reliable Reporting.

③ conflicting needs :

Stakeholders	conflict
→ customer & payment providers	Security vs convenience
→ customer & service provider	Refund policy
→ customer & system admin	Real-Time update

④ software developers

Business requirements, regulatory compliance, Technology constraints, Budget & deadlines.

Task 3 :

FR-ID

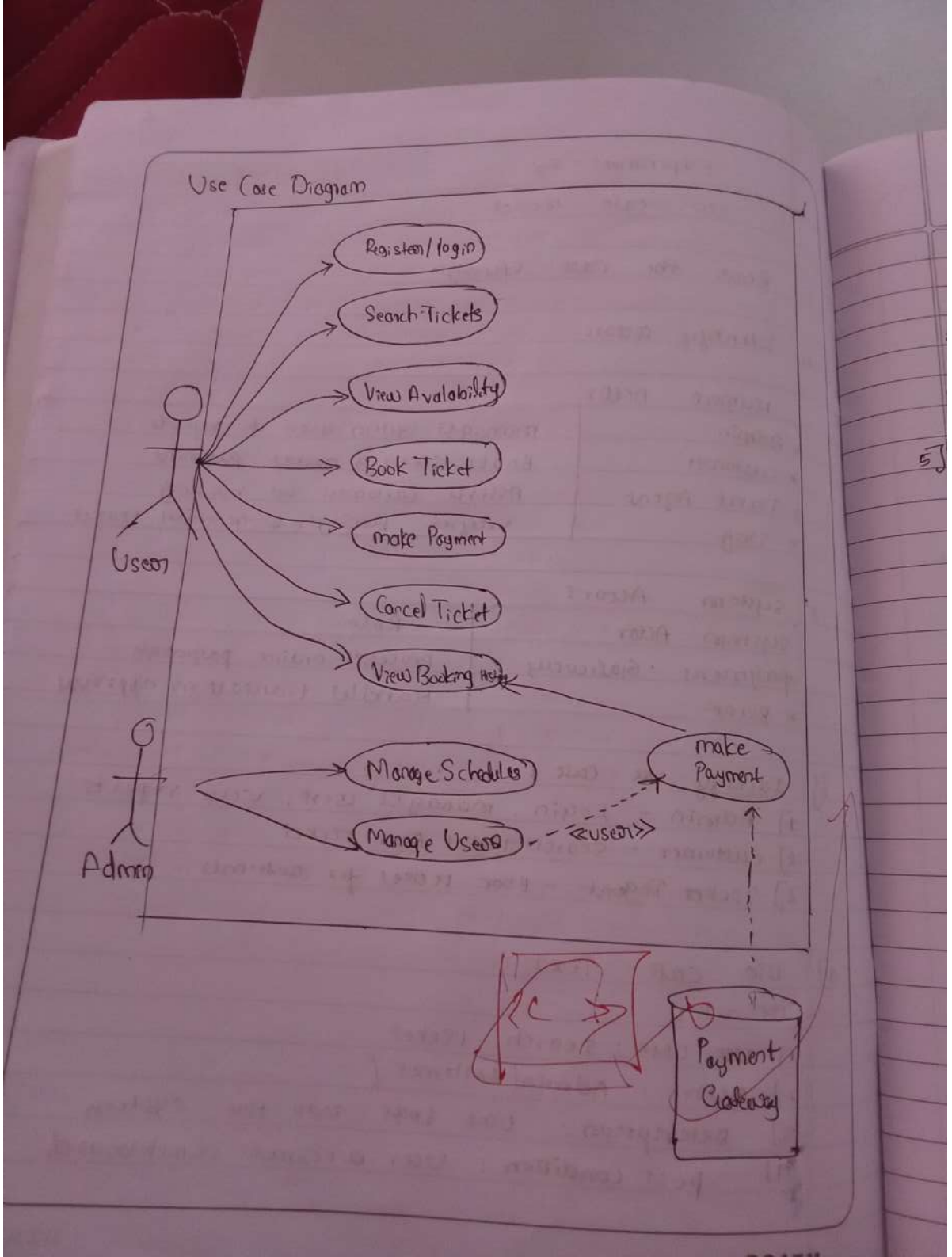
FR-01

FR02

FR-03

Task 3 : Functional Requirements

FR-ID	Descriptn	priority	source	Acceptance criteria
FR-01	User Registration	High	Customer	User account created successfully with verificatn
FR-02	User Login	High	Customer	valid login grants access : invalid error
FR-03	Search Tickets	High	Customer	System display available ticket based on input
FR-04	seat availability	High	Customer	Real-time seat status displayed correctly
FR-05	Book Ticket	High	Customer	Booking ID generated after seat selection
FR-06	payment processing	High	payment Gateway	payment success / failure
FR-07	Booking confirmation	High	Customer	confirmation email / sms sent after payment
FR-08	Cancel Ticket	High	Customer	Ticket cancelled
FR-09	Refund processing	High	Finance	Refund initiated as per policy
FR-10	manage schedules	High	Admin	Admin can add / edit & delete
FR-11	Generate Reports	medium	manage-ment	System generates downloadable Reports
FR-12	Tax calculation	High	Tax Authority	GST calculation


Use Case Diagram
Register/login
Search Tickets
View Availability
Book Ticket
make Payment
Cancel Ticket
View Booking
User
Manage Schedules
Manage Users
Admin
make Payment
≪user≫
Payment Gateway

Experiment - 03 d d m m y y y y

Use case model.

1. Read the case study:

2] Identify actors:

Human Actor	Role
* Admin	manages system, users & reports
* Customer	Books tickets & makes payment
* Ticket Agent	Assists customers in booking
* Staff	verifies bookings & generates ticket

3 System Actors

System Actor	Role
*payment Gateway	processes online payments
* Bank	Handles transaction approval

3] Identify use case:

1] Admin - Login, manages users, view reports
2] Customer - Search ticket, Book ticket
3] Ticket Agent - Book ticket for customer.

4] Use case Text:

UC - 01

1] Use Case: search ticket
2] Actor: Admin/Customer
3] Description: User logs into the system
4] post condition: User accesses dashboard

d d m m y y y y

4] use case text
UC-02
1] use case : search ticket
* Actor : customer
* Description : customer searches for available tickets
* postcondition : Available ticket are displayed.

5] Fully Dressed use case
UC-03
US-Name : Book Ticket
* Scope : Online ticket Booking system
* Level : user level
* primary Actor : customer
* stakeholders : customer

precondition :
* User must be logged in
* Tickets must be available

Main Success Scenario :
1. customer search for tickets
2. select desired ticket
3. enter passenger details
4. confirm booking
5. make payment.

7] main success scenario :-

1. Customer searches for ticket
2. Customer selects ticket
3. System shows total amount
4. Customer makes payment
5. System confirms bookings
6. System generate ticket
7. System sends confirmation.

extension :-

If ticket are not available the system shows "No tickets available".

8] show include & exclude Relationship :

* Book ticket <> search ticket
* Book ticket <> make payment
* Book ticket <> Generate ticket
* manage Booking <> cancel ticket
* manage Booking <> view Booking
* Book ticket <> Apply Discount
* Payment <> Refund payment.

d d m m y y y y

* <<Include / Extend>>

* Book ticket <> make payment
* Generate ticket <> Booking confirmation
* Book Ticket <> cancel ticket
* payment <> Refund

* Deliverable :-

Scope :

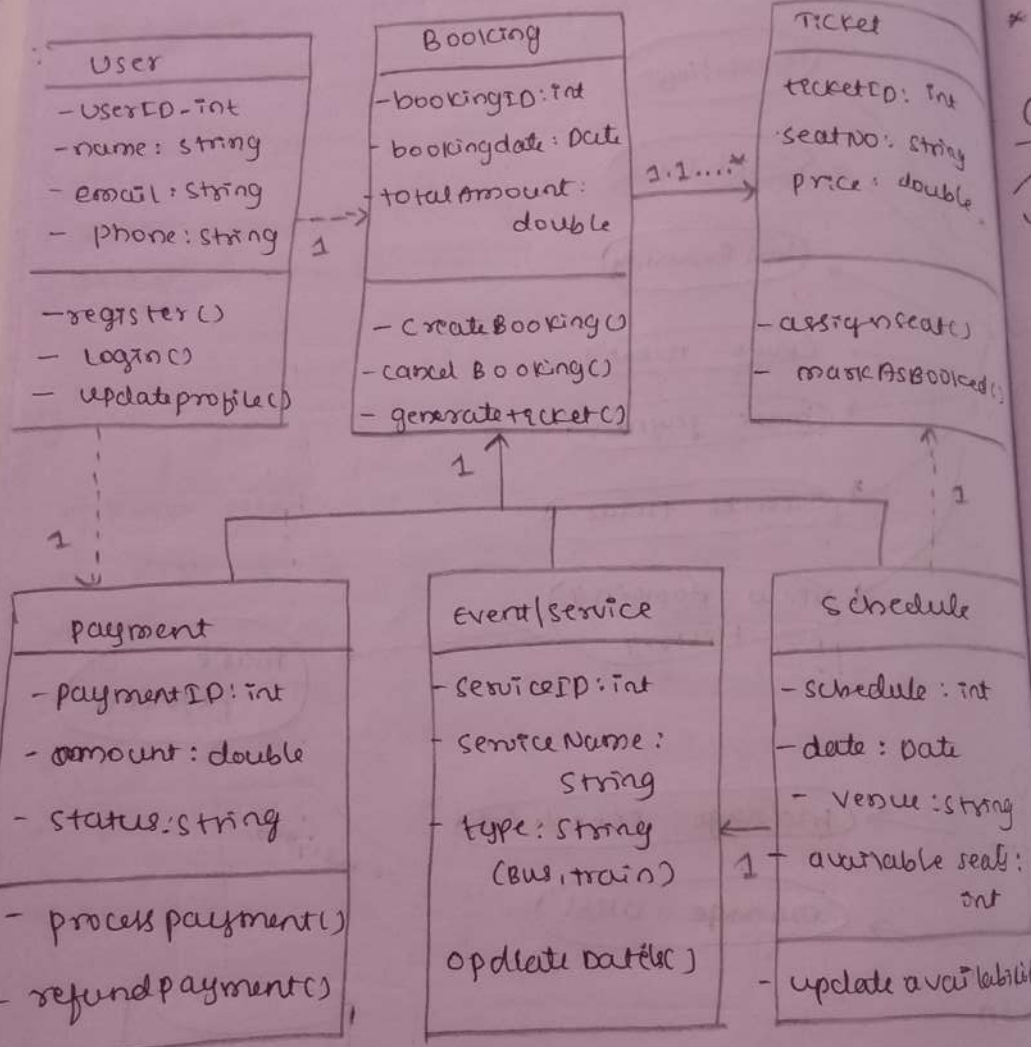
The system allow users to search, book & manage tickets online. It enabled admin to manage users, monitors bookings & generate reports. the system also supports secure payment.

* Use case list :-

1. Login
2. manage users
3. search ticket
4. Book ticket
5. cancel Ticket
6. make payment
7. Generate ticket
8. send Notification.
9. view Reports.

4/4

4] Design Class Diagram



5] SRS

* syste

user

(1)

d d m m y y y y

Experiment - 04.

Drawing diagram using the draw.io class

main classes :-

* user (userID, name, role, email)
* customer (customer ID, name, Phone)
* Ticket (ticket ID, price, seatnumber)
* Booking (BookingID, date, status)
* Admin (adminID, name, role)
* payment (payment ID, amount, status)

* Relationships :-

* Customer (1) ⟶ (M) Booking
* Booking (1) ⟶ (M) Ticket
* Ticket (M) ⟶ (1) Schedule
* Admin (1) ⟶ (M) Schedule.
* user (1) ⟶ (1) customer | Admin

Experiment - 05

d d m m y y y y

SRS (Software Requirement specification)

1. Introduction

1.1 purpose :-
The purpose of the document is to decorate the requirement of the systems

1.2 Scope :-
The system manage product, stock, billing & payment in a super market.

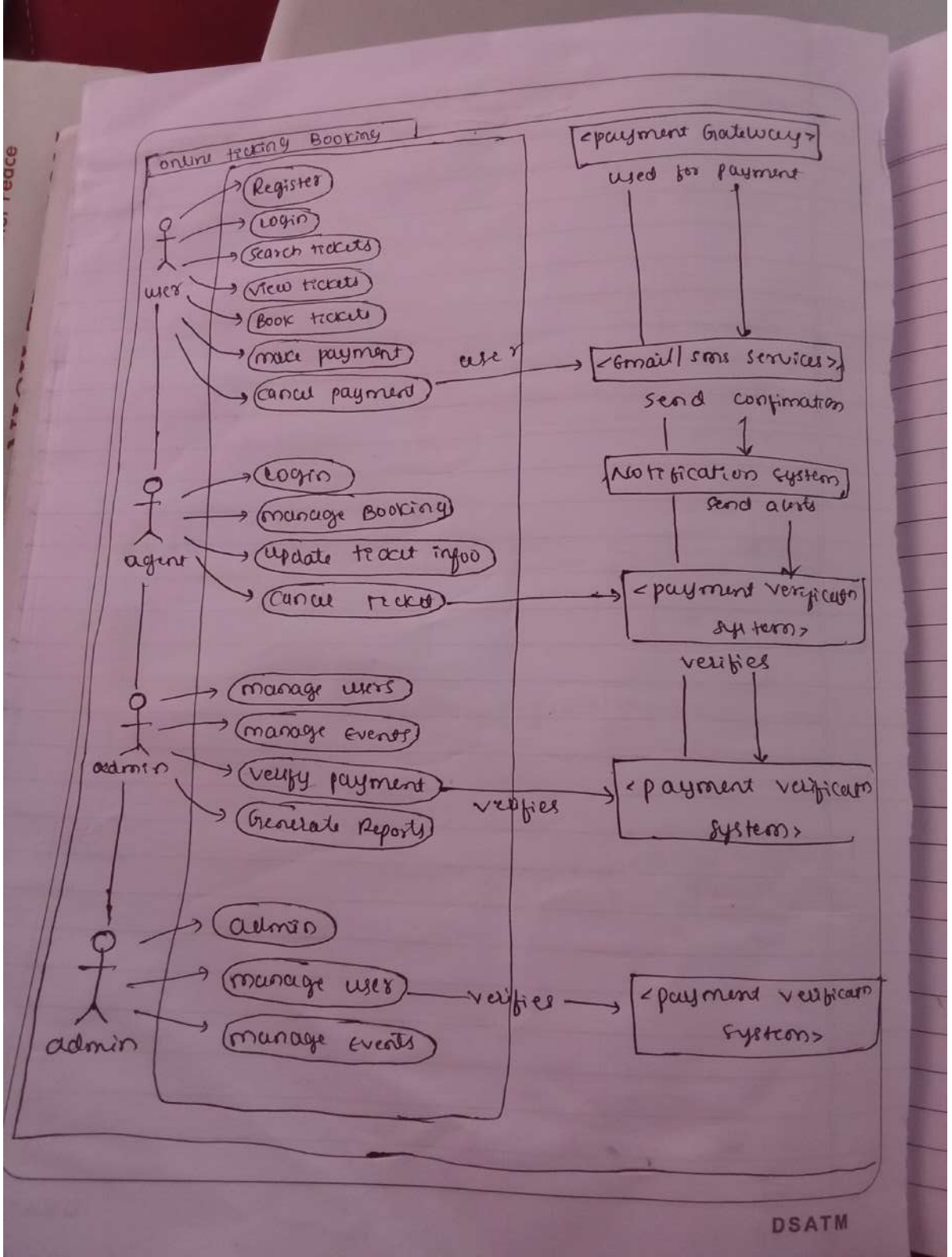
2. overall discription :-

2.1. product perspective :-

The system is stand alone application used in inventory.

2.2 product functions :-

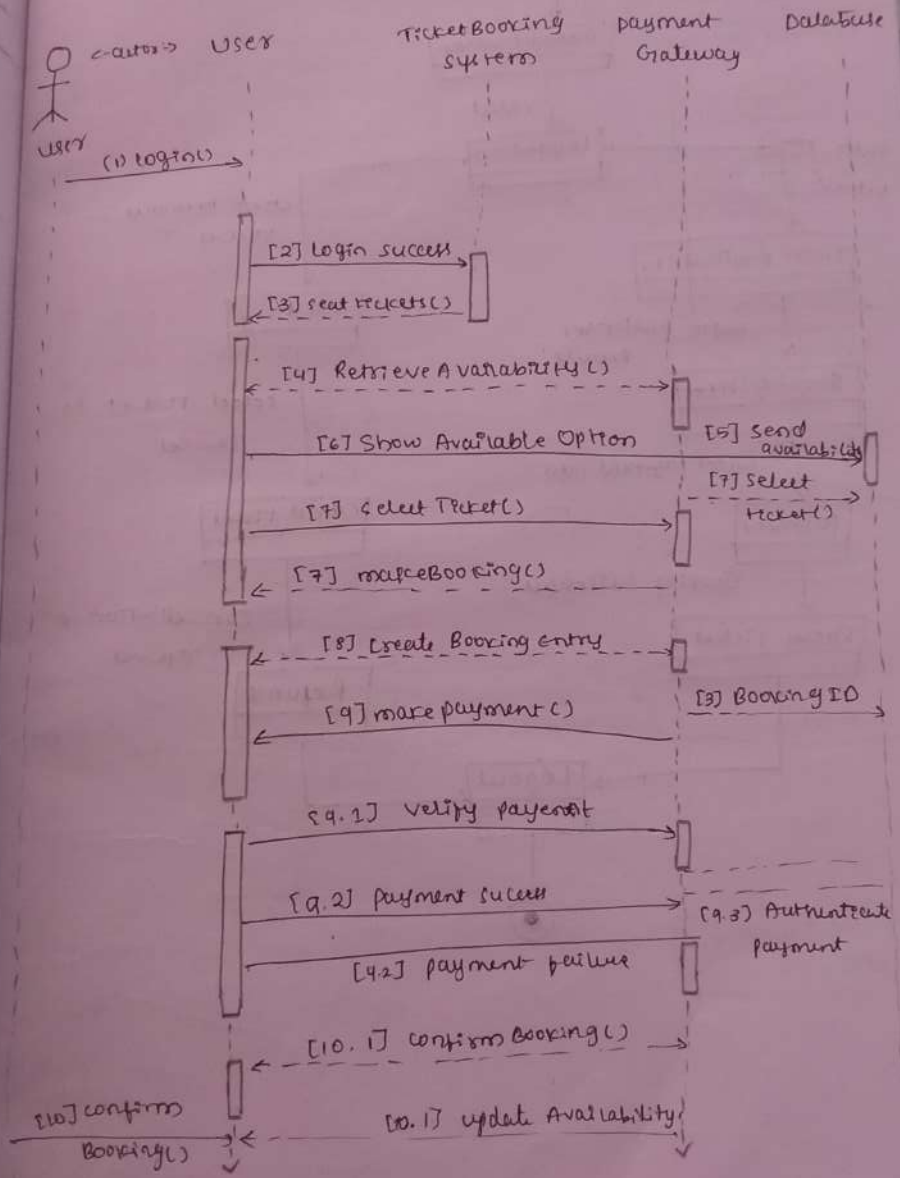
* manage tickets
* update ~~sta~~ ticket
* generate payment


Online Ticking Booking
Register
Login
Search tickets
View tickets
Book tickets
make payment
cancel payment
user
Login
manage Booking
Update ticket info
Cancel ticket
agent
manage users
manage events
verify payment
Generate Reports
admin
admin
manage user
manage events
admin
<payment Gateway>
used for payment
user
<Gmail/sms services>
send confirmation
Notification system
send alerts
<payment verification system>
verifies
verifies
<payment verification system>
verifies
<payment verification system>

DSATM

5] SRS (template)

* System sequence Diagram :-



d d m m y y y y

2.3 User classes :-

* Admin
* customer
* Ticket Agent
* payment Gateway System

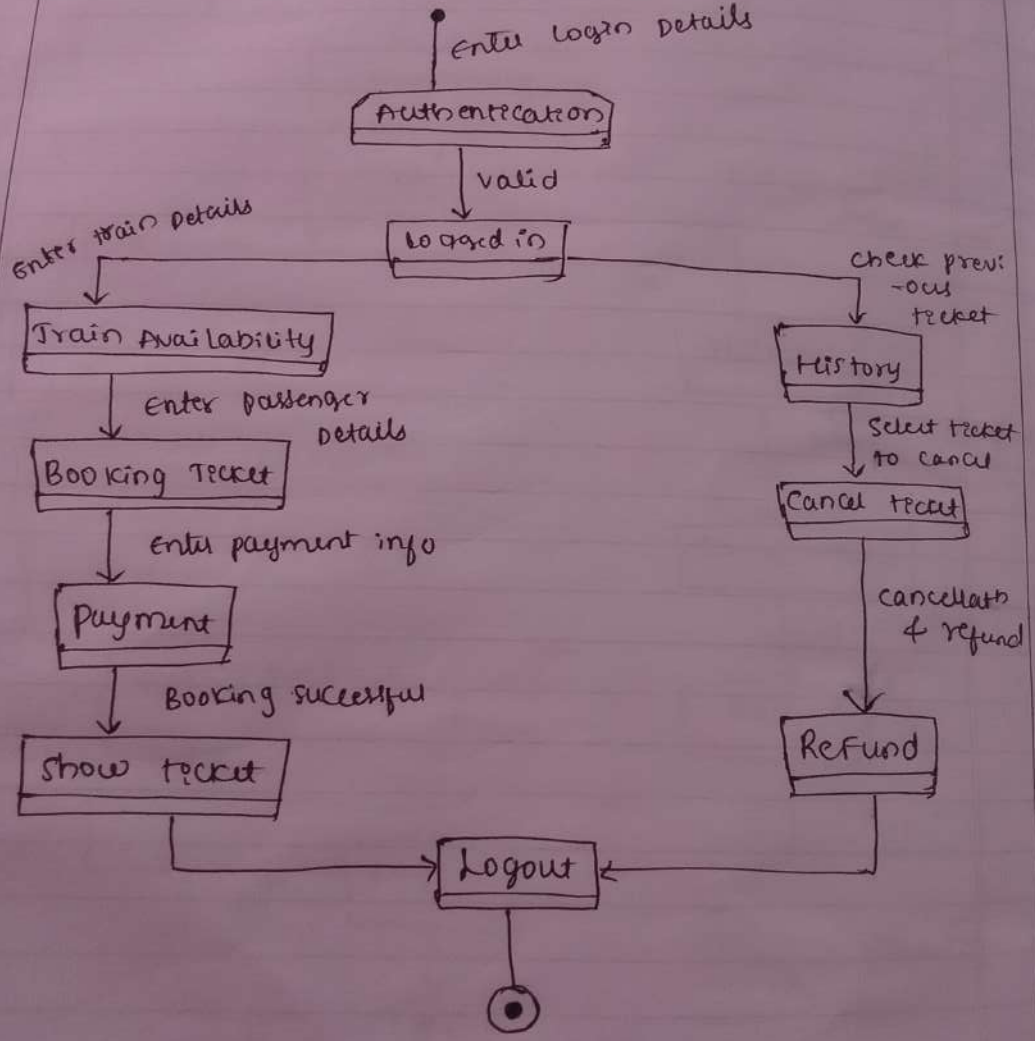
3]: Functional Requirements :-

* System shall allow user to register & login
* system shall allow user to select seat & book ticket
* allow for make Online payments.
* system allow user to cancel ticket.

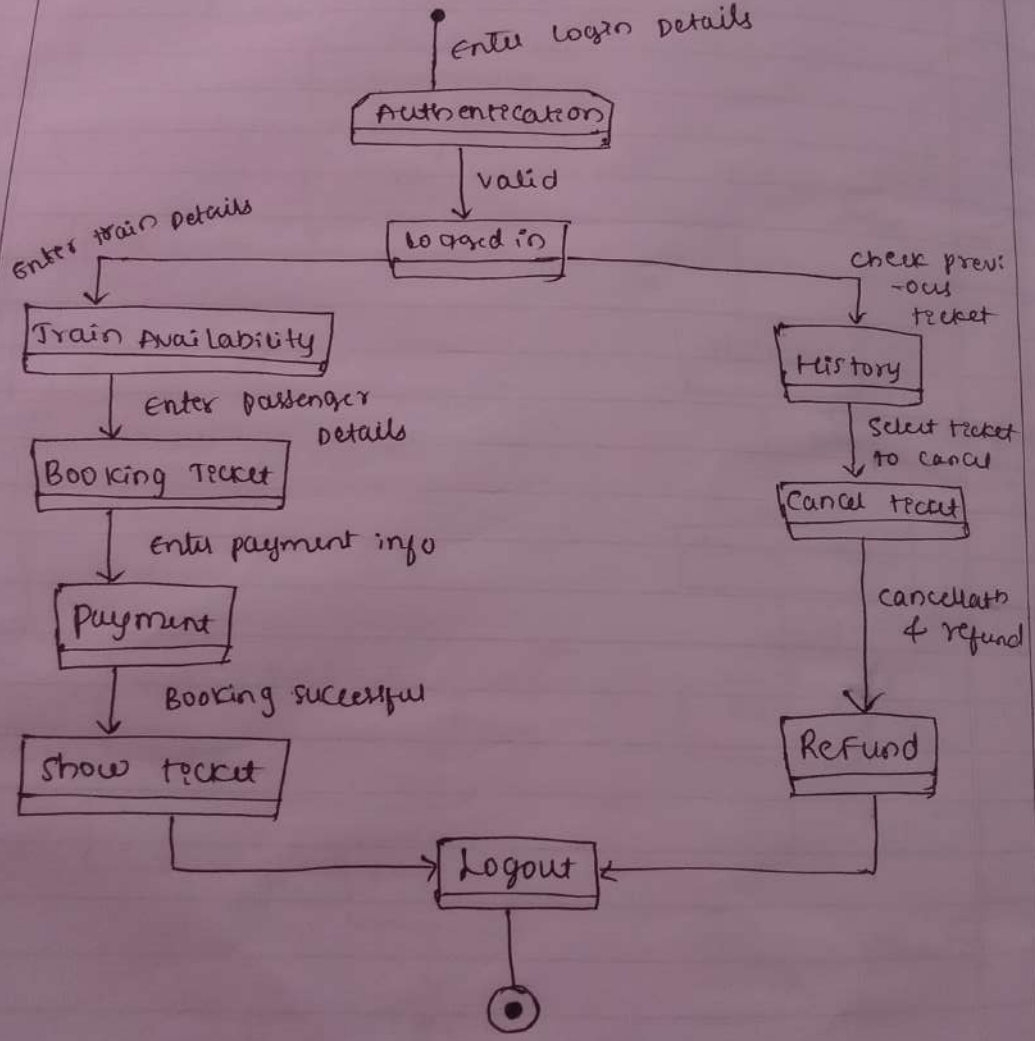
4] Non- Functional Requirements :-

* Security : secure login & payment
* Usability :- user - friendly
* Scalability :- Handle multiple user at the same time
* Availability :- minimal downtime

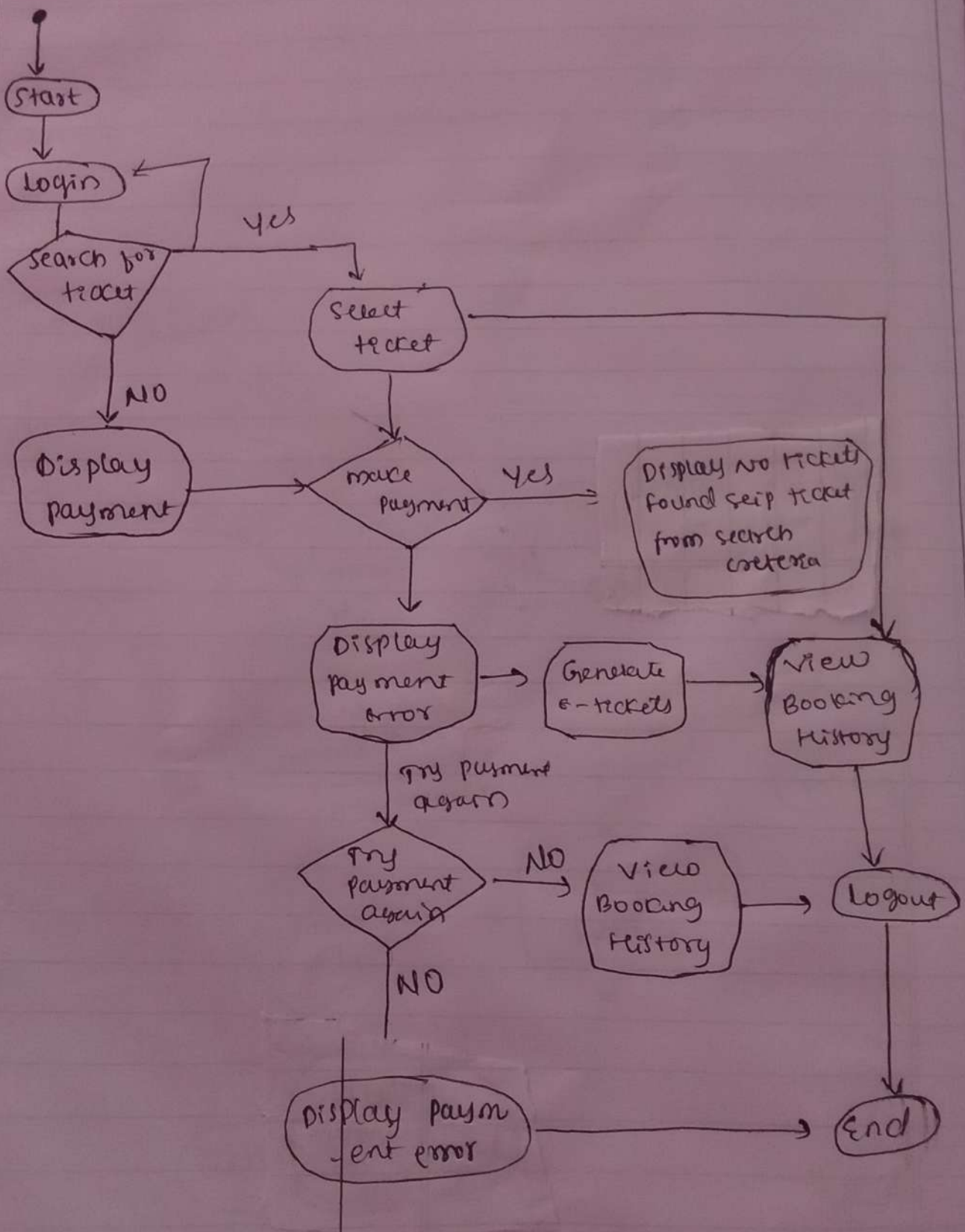
* State chart Diagram :-



* State chart Diagram :-



* Activity Diagram :-



Experiment -06

d d m m y y y y

Configuration management & version control

(CM): managing changes in software files
Types: SVN, Git

Tools:
Git - version control system
GitHub - Remote hosting platform

Commands:
git init
git add
git commit -m 'message'
git checkout <commit-id>
git push origin main

Lab steps:

Initialize repository: git init
create file.txt

Add file:
git add file.txt

commit: git commit -m "Hello git"

d d m m y y y y

git config -- global user.name "Ankita"

git config --global user.email "ankitasuryawanshi290-3@gmail.com"

git log

Author : Ankita < ankitasuryawanshi2903@gmail.com >

Date : Thu Apr 23

Hello Git

Experiment – 07

d d m m y y y y

Design & Development of a system using Design pattern & SDLC process

Aim:-

To Design & Develop an online ticket booking system by applying SDLC phases.

Team Roles:-

- Customer: provides requirements for ticket booking system
- Developer 1: Deplements ticket search & booking system using design pattern.
- Developer 2: Implement user management & payment system

Requirement phases:-

2] Design phases:-

- Design classes
- Implement ticket search system
- Implement booking system
- Test the system

Identified classes:-

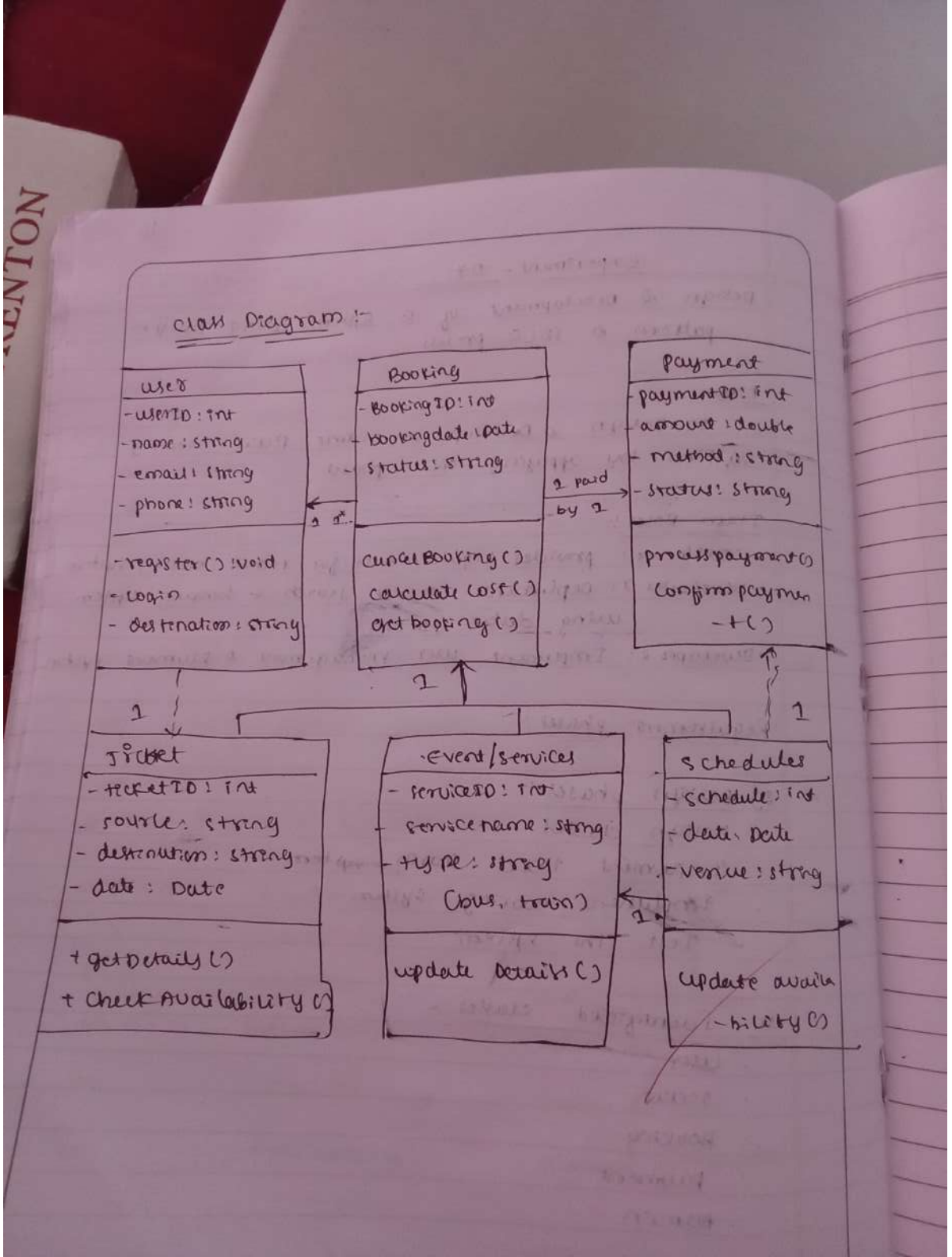
- User
- Ticket
- Booking
- payment
- Admin

Requirement phases :-

FR-ID	Functional Requirement	priority	source	Acceptance Criteria
01	user registration & login	High	customer	valid login
02	search tickets	High	customer	search result shown
03	view ticket details	High	Customer	Details Displayed
04	BOOK ticket	High	customer	seat booked
05	online payment	High	customer	payment confirmed.

Non-functional Requirements :-

NFR-ID	Requirements	Description
01	performance	fast response
02	security	Data protection
03	Reliability	Error-free system
04	Usability	Easy to use
05	Availability	Always accessible


Class Diagram :-
User
- userID : int
- name : string
- email : string
- phone : string
- register () : void
- login
- destination : string
Booking
- BookingID : int
- bookingdate : date
- status : string
CancelBooking ()
calculate cost ()
Get Booking ()
1 paid by 1
Payment
- paymentID : int
- amount : double
- method : string
- status : string
processpayment ()
Confirm paymen
-t ()
Ticket
- TicketID : int
- source : string
- destination : string
- date : Date
+ getDetails ()
+ CheckAvailability ()
Event/services
- serviceID : int
- service name : string
- type : string
(bus, train)
update Details ()
Schedules
- schedule : int
- date : Date
- venue : string
Update availa
-bility ()
1
1
1
1
1

d d m m y y y y

Class Responsibilities :

- User : Register, login, search ticket, book ticket
- Ticket : Stores ticket details
- Booking : Handles booking & seat
- payment : Handles payment processing

3] Implementation phase

- Ticket search
- Booking creation
- Cost calculation
- payment processing

Output Example :

- Ticket selected : Bang → Bidar
- Ticket total : 1200 ₹
- Booking confirmed

4] Testing phase :

- Tested booking functionality
- verified payment proces
- Checked ticket availability
- validated multiple booking

5] Deployment phase :

- System executed in lab experiment
- Demonstration Complemented

DSATM

Experiment - 08

d d m m y y y y

Apply Scrum - Based Agile Methodology

Aim :-

Apply scrum - based Agile methodology & implementation an online ticket booking system & test a design pattern

Roles :-

- Observer : monitors process & record activities
- Customer : provides requirements
- Developer : Design UML & implement system
- Testers : tests system & verifies output

problem statement :

- Allow users to search ticket
- Enables booking ticket online
- Calculates total cost automatically

1] product Backlog :-

- As a user, I want to search ticket so that I can plan my travel
- As a user, I want to book tickets so that I can confirm my journey
- As a user, I want to see booking status updates.
- As a developer, I want loosely coupled components

DSATM

d d m m y y y y

booking_system = TicketBooking()
user_display = userDisplay()
admin_display = adminDisplay()

booking = {
    "route": "Bangalore - Bidar",
    "seats": 2,
    "price": 800,
}
calc = CostCalculator()
total = calc.calculate_total(booking["price"],
    booking["seats"])
booking["total"] = total

booking_system.StatBooking(booking).

5] Testing :-

Input :-

package : Bangalore - Bidar
people : 2
total : 4200 ₹

output :-

parking updated : {'package', 'Bangalore - Bidar', 'people': 2,
'total': 4200}

user view : property confirmed for Bang - Bidar

Admin view : Booking updated - {'package', 'Bang-
Bidar', 'People': 2, 'total': 4200}

d d m m y y y y

6] Sprint Review :-

- Booking system orders successfully
- total cost calculated
- Notification - displayed

7] Retrospective :-

- Easy Implementation
- clear separation of logic
- Real-time update.

Experiment - 09 d d m m y y y y

Standard project plan Template

project Name: library management system
Domain: Education / information management services

1. project overview

purpose:

To develop a centralized web-based platform that automates book catalog management, member registration, book issuing/returning, fine calculation & report.

mission statement:-

To provide an efficient an user-friendly library experience by digitizing manual library operations into a single integration system.

Objectives:

• Implement a modular architecture using 8 functional cel package for better maintainability

project-scope:

* In-scope:
  - user registration & profile management
  - Book Katalog browsing & search
  - fine calculation & payment tracking

* out of scope:
  - physical book delivery service
  - Inter-library book exchange system.

DSATM

d d m m y y y y

2] Team organization & roles & stakeholders:
* product owner:
→ Responsible for defining business requirement & backlog prioritization.
* scrum master:
→ Responsible for maintaing sprint schedule, ensuring Agile compliance, & remove
* Development team:
→ Responsible for UML Design, Java/Python

3] work Breakdown structure (WBS):
1] Requirements Analysis:
→ Identify user stories, define acceptance criteria, & build product backlog.
2] System Design:
→ Create UML Design including use case, class sequence Activity & SC.
3] Implementation:
→ Develop core module using Java, applying Factory Design pattern.
4] Testing:
→ Write & execute test cases for functional verification & performance.

4] project schedule (Agile sprints) :-
* Sprints 1:- Focus on BOOK & search package, development of browsing & search features

d d m m y y y y

* Sprints 2 (Issue & Returns) :-
Focus on transaction & member Authentication package logic for Issue & return

* Sprints 3 (Fine & Reports) :-
Focus payment & Notification on package

5] Configuration management plan :-
* Branching Strategy :-
Features - based branching to allow isolated

* Code Reviews :
mandatory Pull Request reviews to ensures code

6] Quality Assurance & Testing :-
* Unit testing :-
→ Validation individual classes such as Book, members & transaction "

* Integration testing :
→ verifying communication btw issues return module & fine

* Succes metric :-
→ 100% of user stories in sprint backlog

DSATM

d d m m y y y y

7] Risk management :-

Risk	Description	impact	migration strategy
* Technical	Database corrupt -due failure	High	implement backup data -back error.
* Managerial -risk	Scope creep during Sprint	medium	strict adherence to define backlog
* Security	unauthorized access to records	high	use password hasing & role based on access.

8] project closure & Deliverables :

(i) Software Requirements specification (SRS) :
Full list of FR & Non-FR.

(ii) UML Design Document :
All 7 dia showing static & dynamic system.

(iii) Working Prototype :
Source code implementation of core library.

(iv) Agile artifact :
Documentation backlog, sprints plan & retrospective

DSATM

Experiment - 10

d d m m y y y y

Develop test cases and use the same for system testing software. Generate test reports

System Testing & Test Driven Development (TDD) Report.

1. Test Driven Development (TDD) methodology.
It follows three-stages TDD life cycle.

1] Red phase (write test first): We write a test case for a feature before coding it. The test fail initially bcz the feature doesn't exist.

2] Green phase (write code to Toss):
We write the simplest functional code possible to make that test pass.

3] Refactor phase (Improve code):
We design & optimize the code while ensuring

2. system test suite (Test case 2)

Test case ID	Feature tested	Input	ExpectResult	Status
TS_TC_01	Factory create of Flight Booking	TYPE'AIR -HT'	valid Flight Booking object	pass
TS-TC-02	total Intinerary Collection	Flight A (30) Flight B (15.00)	Cumulative total to exactly	pass
TS-TC-03	Invalid input coding	TYPE: TRAIN	Return null on handled exception	pass

DSATM

d d m m y y y y

3. TDD Implementation code & Refactoring

Step 3.1 : The failing Test (Red phase)
public class written BEFORE the actual logic exists

```
public class BookingTest {
    public static void main(String[] args) {
        Booking booking = new Booking();
        booking.addFlight(new Flight("FL101",
                                     500.00));
        double finalPrice = booking.applyDiscount("SAVE10");
        System.out.println(finalPrice == 450.00? "pass:
                                                  fail");
    }
}
```

Step 3.2 : writting simple code to pass (Green phase)

```
class Booking {
    List<Flight> flights = new ArrayList<>();
    public void addFlight(Flight f)
    public double getTotal() {
        double total = 0;
        for(Flight f: flights) { total += f.price;}
        return total;
    }
    public double applyDiscount(String promoCode) {
        if("SAVE10".equals(promoCode)) return getTotal()
                                              0.90;
        return getTotal();
    }
}
```

DSATM

d d m m y y y y

Step 3.3 : Code Cleanup (Refactor)

```
class Booking {
    private List<Flight> flights = new ArrayList<>();
    public void addFlight(Flight flight) {
        if(flight != null) this.flights.add(flight);
    }
    public double getTotal() {
        return flights.stream().mapToDouble(f -> f.price).
                                                sum();
    }
    public double applyDiscount(String promoCode) {
        if("SAVE10".equalsIgnoreCase(promoCode)) {
            return getTotal() * 0.90;
        }
        return getTotal();
    }
}
```

4 System test summary

- Total test Run : 3
- passed : 3
- Failed : 0
- code coverage : 100% of cake classes in the sprint Backlog
- Conclusion : using TDD kept our codebase highly modular and kept the project completly free of common Runtime bugs.

DSATM

Task 4 : Non-functional Requirements

NFR-ID	Descriptn	priority	sou ce	Acceptanc criteria
NFR-01	performance	High	customer	Response time ≤ 3 sec
NFR-02	Availability	High	management	Uptime ≥ 99%
NFR-03	security	High	Regulator/IT	HTTPS enabled, data encrypted
NFR-04	Reliability	High	Service prov -ider	No duplicate bookings.
NFR-05	Scalability	medium	manageme -nt	Handles peak users without crash.
NFR-06	Usability	medium	customer	Booking completed in ≤ 5 steps
NFR-07	Compatibi -lity	medium	customer	Works on web & mobile browsers
NFR-08	Backup & Recovery	High	IT Team	Data restored within defined time
NFR-09	maintaina -bility	medium	Developer	modular, docu -mented code
NFR 10	compliance	High	Regulatory	meets privacy & tax rules.

Task 5 : Describe your case study.

what it is : online system to book for movies, trains & buses

why it's need : Easy, fast booking, online payment

who use it :

→ customer & Admin 26/3/26

How it work : Login → choose event → pick seat → pay → Get e- ef